

認証機能 実装フェーズ

JWT + httpOnly Cookie によるログイン基盤の構築

Hyonun Kin

2026-04-27

目的・背景

文章校正サービスはユーザー単位で履歴を保持するため、リクエスト毎に「誰がアクセスしているか」を確実に識別する必要がある。一方で、社内向けの小規模サービスのため、運用コストとセキュリティのバランスが取れた仕組みを採用したい。

そこで本フェーズでは **JWT を httpOnly Cookie に格納する方式** を採用し、後段の校正 API と管理者 API で共通利用できる認証基盤を構築する。

要件

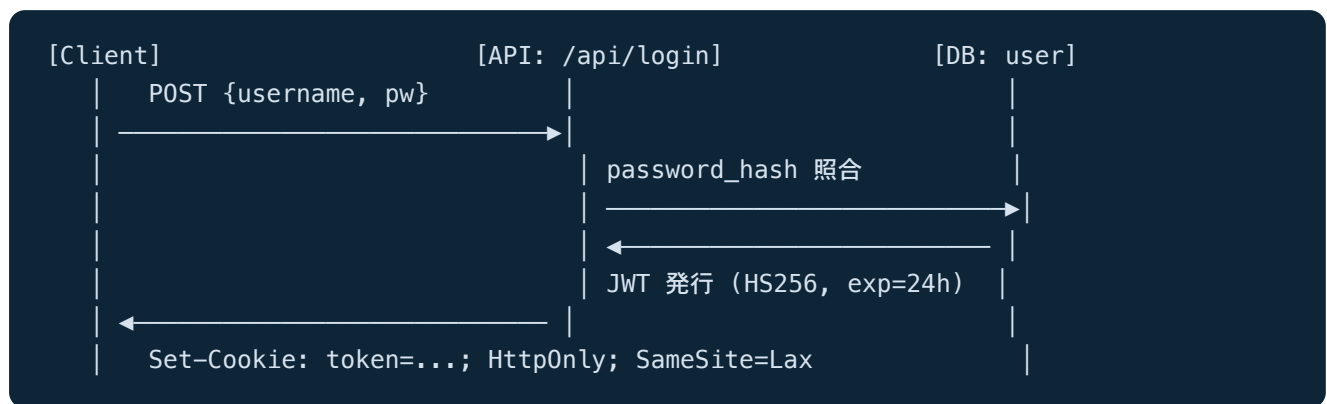
機能要件として以下を満たす。

- ID / パスワードでのログインができる
- ログアウトで Cookie を破棄できる
- 認証必須エンドポイントは Cookie の JWT を検証してから処理する
- 管理者ロール (admin) と一般ロール (user) を区別できる

非機能要件として、SameSite 属性で CSRF を防ぎ、Secret は環境変数から読み込む。

設計

認証フロー



ログイン成功後は Cookie がブラウザに保存され、以降のリクエストで自動付与される。サーバー側はミドルウェアで `token` Cookie を読み、`jwt.decode()` で検証する。

JWT クレーム構造

クレーム	型	内容
<code>sub</code>	<code>int</code>	<code>user.id</code>
<code>username</code>	<code>str</code>	表示用
<code>role</code>	<code>str</code>	<code>user</code> または <code>admin</code>
<code>exp</code>	<code>int</code>	発行から 24 時間

提供エンドポイント

- `POST /api/login` — 認証してCookieを発行
- `POST /api/logout` — Cookieを失効させる
- `GET /api/me` — 現在のログインユーザー情報を返す（疎通確認用）

実装

ファイル構成

本フェーズで追加・変更したファイルは以下の通り。

```
backend/api/
├── auth.py          # JWT 発行 / 検証 / 依存関数
├── models.py        # User テーブル
├── session.py        # AsyncSession
└── app.py           # /api/login, /api/logout, /api/me を追加
```

主要コード (auth.py 抜粋)

```
SECRET = os.environ["JWT_SECRET"]
ALGORITHM = "HS256"
EXPIRE_HOURS = 24

def create_token(user: User) -> str:
    payload = {
        "sub": user.id,
        "username": user.username,
        "role": user.role,
        "exp": datetime.utcnow() + timedelta(hours=EXPIRE_HOURS),
    }
    return jwt.encode(payload, SECRET, algorithm=ALGORITHM)

async def current_user(
    token: str | None = Cookie(default=None),
    session: AsyncSession = Depends(get_session),
) -> User:
    if token is None:
        raise HTTPException(401, "not authenticated")
    try:
        payload = jwt.decode(token, SECRET, algorithms=[ALGORITHM])
    except jwt.PyJWTError:
        raise HTTPException(401, "invalid token")
    user = await session.get(User, payload["sub"])
    if user is None or not user.is_active:
        raise HTTPException(401, "user not found")
    return user
```

`current_user` は FastAPI の `Depends` で各エンドポイントに差し込むことで、認証必須ルートは依存関係を一行追加するだけで保護される。

Cookie の付与方針

- `httponly=True` — JavaScript からのアクセスを禁止し XSS 対策
- `samesite="lax"` — クロスサイトの POST を抑制し CSRF 対策
- `secure=True` — 本番のみ。HTTPS 経由の通信に限定

開発環境ではブラウザの仕様上 `secure=False` で動かすため、設定は環境変数 `COOKIE_SECURE` で切り替える。

検証

ローカルでの動作確認を以下の順で実施した。

- `uv run pytest tests/test_auth.py` — 7 ケースすべて pass
- curl での疎通確認
 - 有効な ID/PW でログイン → `Set-Cookie` が返る
 - Cookie 無しで `/api/me` → 401
 - Cookie 付きで `/api/me` → ユーザー情報が返る
 - `/api/logout` 後に `/api/me` → 401

テストの観点

```
async def test_login_sets_cookie(client, taro):
    res = await client.post("/api/login", json={
        "username": "taro", "password": "taro1234",
    })
    assert res.status_code == 200
    assert "token" in res.cookies
    assert res.cookies.get("token") # HttpOnly なのでヘッダで再確認

async def test_protected_requires_cookie(client):
    res = await client.get("/api/me")
    assert res.status_code == 401
```

次フェーズへの引き継ぎ

本フェーズで認証基盤が整ったため、次の **Step 3: 校正API** では `current_user` 依存を 各エンドポイントに差し込むだけで利用ユーザーを特定できる。

- `POST /api/correct` — `current_user` を依存に追加し、`corrected.user_id` に紐づける
- `POST /api/admin/user` — `current_user.role == "admin"` のガードを追加する

注意点として、JWT の有効期限は 24 時間に設定されているため、長時間放置されたタブで失効するケースをフロント側でハンドリングする必要がある（401 受信時にログイン画面ヘリダイレクト）。